



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Uniform-Cost Search (UCS) prioritizes nodes based only on the actual path cost from the start node:

$$f(n) = g(n)$$

A* Search considers both the actual cost and an estimated cost to the goal:

$$f(n) = g(n) + h(n)$$

Therefore, UCS explores the cheapest path found so far, while A* uses the heuristic to prioritize nodes that appear to be closer to the goal. This allows A* to generally explore fewer unnecessary nodes while still finding an optimal path when the heuristic is admissible.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Manhattan Distance is admissible for a grid where the agent can only move **Up, Down, Left, or Right** because it never overestimates the actual minimum number of moves required to reach the goal.

It is calculated as:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

For example, if the agent is at $(0,0)$ and the goal is $(3,4)$, the Manhattan distance is:

$$|0 - 3| + |0 - 4| = 3 + 4 = 7$$

At least 7 horizontal/vertical moves are required when there are no obstacles, so the heuristic is optimistic. Obstacles can only make the actual path longer, never shorter.

If the heuristic is **not admissible** and overestimates the actual cost, A^* may choose a more expensive path because it incorrectly believes the optimal path is more costly. Therefore, *A is no longer guaranteed to find the optimal solution*.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No. **Manhattan Distance would no longer be admissible** if diagonal movement is allowed and diagonal movement has the same cost as horizontal/vertical movement.

For example, from $(0,0)$ to $(3,3)$:

- Manhattan Distance = $3 + 3 = 6$
- With diagonal movement, the agent can reach the goal in only **3 diagonal moves**.

Therefore, Manhattan Distance would overestimate the actual cost, violating the admissibility requirement.

For an 8-way grid where diagonal and straight movements have equal cost, a suitable heuristic is **Chebyshev Distance**:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

If diagonal movement has a different cost from straight movement, the heuristic should instead account for those different movement costs.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Using only the distance to the **closest food item** is weak because it considers only the next target and ignores the other food items that still need to be collected.

A stronger heuristic would estimate the **minimum total cost required to visit all remaining food items**. One approach is to calculate the distances between the current position and all remaining food locations, as well as the distances between the food locations themselves, and use a **Minimum Spanning Tree (MST)** over the food positions.

The heuristic could be:

$h(n)$ = distance from current position to the food set + MST cost connecting all remaining food

This provides a stronger estimate of the cost required to eat all food while remaining optimistic. It considers the relationships between **all remaining food items**, rather than only selecting the closest one, and can therefore guide A* toward a more efficient overall route.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING